

Lab5：进程及进程管理

本章完成的工作

本章完成了进程管理与调度机制的理解和编程作业：

- 1. 成功运行 ch5 代码，体验全新的交互式 Shell
- 2. 理解进程创建机制：fork/exec/wait 三大系统调用
- 3. 理解进程状态转换，特别是 ZOMBIE 状态的作用
- 4. 深入理解 Stride 调度算法的原理与实现
- 5. 实现 sys_spawn 系统调用（编程作业）
- 6. 实现 sys_set_priority 系统调用（编程作业）
- 7. 实现 Stride 调度算法（编程作业）
- 8. 修复 freewalk 函数以支持 mmap 区域清理

本章有编程作业：实现 spawn 系统调用和基于优先级的 Stride 调度算法。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
本章导读（Shell介绍）	第二节：初见交互式Shell	体现"第一次见到Shell"的惊喜与学习过程
与进程相关的系统调用	第三节：进程系统调用深入理解	从"不懂fork返回两次"到理解的过程
进程管理的核心数据结构	第四节：进程调度机制	增加FIFO vs Stride的对比分析
Shell与测例加载	第五节：Shell工作原理分析	详细分析usershell.c的实现
chapter5练习	第六节：编程作业实现	记录stride调度的实现思路和遇到的问题

本报告的特点：

- 1. 强调交互体验的变化：从Lab4的"自动执行测例"到Lab5的"手动输入命令"，这是用户体验的巨大飞跃
- 2. 体现学习探索过程：特别是对fork"一次调用返回两次"的理解
- 3. 增加可视化内容：进程状态转换图、Stride调度示例图
- 4. 记录真实问题：freewalk panic问题与ch4相同的根本原因

一、实验环境与运行

1.1 代码目录结构

2025-ucore-riscv-清华/
├─ uCore-Tutorial-Code-2025S-ch5/
│ ├─ os/
│ │ ├─ proc.c
│ │ ├─ proc.h
│ │ └─ syscall.c
│ │ └─ vm.c
│ │ └─ queue.c
│ └─ user/
│ └─ src/
│ ├─ usershell.c
│ └─ ch5b_usertest.c
│ └─ ch5t_usertest.c
└─ os-btbu/

← 本章代码

← 内核代码（需要修改）

← 进程管理、spawn、stride调度

← 进程控制块定义

← 系统调用入口

← 虚拟内存（修复freewalk）

← 任务队列

← 用户测试程序

← 交互式Shell（重要！）

← 基础测试

← Stride调度测试

← 最终完成的代码

1.2 本章新增/修改的关键文件

相比 Lab4，本章内核代码的主要变化：

文件	新增/修改	说明
os/proc.c	大幅修改	添加spawn、stride调度、fork/exec/wait实现
os/proc.h	修改	添加stride和priority字段
os/syscall.c	修改	添加sys_spawn、sys_set_priority入口
os/queue.c	新增	任务队列实现
os/loader.c	修改	load_init_app替代run_all_app

1.3 运行命令与结果

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch5（其他同学可根据实际路径）
make clean
make user CHAPTER=5
make run
```

运行结果（关键部分）：

```
C user shell
>> ch5t_usertest
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
Shell: Process 2 exited with code 0
>>
```

二、初见交互式 Shell：本章最大的惊喜

2.1 从"自动执行"到"手动交互"的转变

运行 `make run` 后，我看到了与之前完全不同的界面：

```
C user shell
>>
```

等等，这是什么？程序没有自动开始执行测例，而是停在了一个 `>>` 提示符处，等待我的输入！

这是我第一次在这个操作系统中体验到**交互式操作**。之前的Lab2-Lab4，所有测例都是内核启动时自动加载、自动执行的，我只是一个"观众"，看着程序跑完。而现在，我成了"操作者"，可以决定运行什么程序！

2.2 尝试交互

我尝试输入 `ch5t_usertest` 并按回车：

```
>> ch5t_usertest
ch5t usertest passed!
ch5t usertest passed!
...
Shell: Process 2 exited with code 0
>>
```

程序执行完毕后，又回到了 `>>` 提示符！我可以继续输入其他命令。这就像Linux的终端一样。

2.3 理解 Shell 的意义

指导书提到：

"我们将实现一个用户终端（Terminal），也称为命令行应用。它会构成用户与操作系统交互的命令行界面。"

现在我深刻理解了这句话：

- **之前**（Lab2-Lab4）：程序是"批处理"模式，用户无法干预
- **现在**（Lab5）：有了Shell，用户可以：
 - 选择运行哪个程序

- 多次运行同一个程序
- 观察每个程序的退出码

这种交互能力是现代操作系统的基础。Windows有CMD，Linux有bash，我们的uCore现在也有了自己的usershell！

2.4 思考：Shell 需要什么能力？

要实现这样的交互式Shell，操作系统需要提供什么能力？

1. **读取用户输入**： `sys_read` 从键盘获取输入
2. **创建新进程执行程序**：Shell本身是一个进程，它需要创建子进程来运行用户指定的程序
3. **等待子进程结束**：Shell需要等待程序运行完毕，获取其退出码，然后继续等待下一条命令

这正是本章要学习的核心内容：进程的创建（fork/spawn）、执行（exec）、等待（wait）。

三、进程系统调用深入理解

3.1 进程状态回顾与新发现

在Lab3中，我已经接触过进程状态：UNUSED、RUNNABLE、RUNNING。但本章我发现了一个新状态：**ZOMBIE**。

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

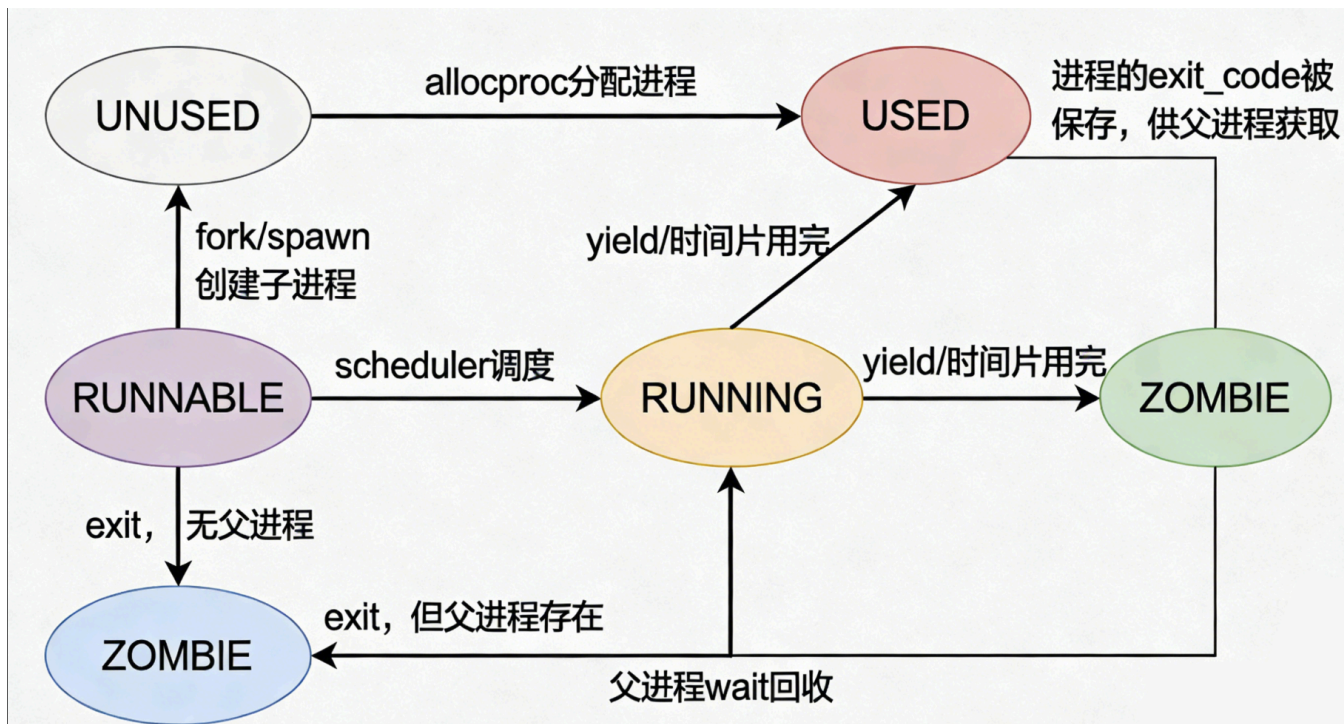
ZOMBIE（僵尸）状态是什么？

刚开始看到这个名字觉得很奇怪——为什么进程会变成“僵尸”？

通过阅读代码和指导书，我理解了：

当一个进程结束时，如果它还有父进程，就不能立即释放所有资源。因为父进程可能需要获取子进程的退出码。所以这个进程进入ZOMBIE状态，等待父进程调用wait来“收尸”。

【原创结构图1：进程状态转换图】



3.2 fork：一次调用，两次返回？

`fork` 是创建新进程的核心系统调用。指导书说：

"fork 创建一个与当前进程几乎完全相同的子进程"

我仔细阅读了 `fork` 的实现：

```
int fork()
{
    struct proc *p = curr_proc();           // 获取当前进程（父进程）
    struct proc *np = allocproc();          // 分配子进程

    // 将父进程的用户内存拷贝到子进程
    uvmcopy(p->pagetable, np->pagetable, p->max_page);
    np->max_page = p->max_page;

    // 复制父进程的trapframe（寄存器状态）
    *(np->trapframe) = *(p->trapframe);

    // 关键：设置子进程中 fork 返回值为 0
    np->trapframe->a0 = 0;

    np->parent = p;
    np->state = RUNNABLE;
    add_task(np);

    return np->pid; // 父进程返回子进程PID
}
```

关键洞察： `fork` 的"一次调用，两次返回"是怎么实现的？

1. 父进程调用 `fork`，执行上述代码，最后 `return np->pid` 返回子进程PID

2. 子进程的 trapframe 被复制自父进程，所以子进程"以为"自己也调用了fork
3. 但子进程的 a0 寄存器被设置为0，所以子进程看到的返回值是0

这样，同一个 fork 调用：

- 在父进程中返回子进程的PID（正数）
- 在子进程中返回0

用户程序可以通过返回值区分自己是父进程还是子进程：

```
int pid = fork();
if (pid == 0) {
    // 子进程执行这里
} else {
    // 父进程执行这里
}
```

3.3 exec：加载新程序

exec 将当前进程替换为一个新程序：

```
int exec(char *name)
{
    int id = get_id_by_name(name);
    if (id < 0)
        return -1;
    struct proc *p = curr_proc();
    uvmunmap(p->pagetable, 0, p->max_page, 1); // 释放旧内存
    p->max_page = 0;
    loader(id, p); // 加载新程序
    return 0;
}
```

关键理解： exec 不创建新进程，而是"换掉"当前进程的程序。进程的PID不变，但执行的代码完全不同了。

3.4 wait：等待子进程结束

wait 让父进程等待子进程结束：

```
int wait(int pid, int *code)
{
    struct proc *p = curr_proc();

    for (;;) {
        // 遍历进程表，寻找已结束的子进程
        int havekids = 0;
        for (struct proc *np = pool; np < &pool[NPROC]; np++) {
            if (np->state != UNUSED && np->parent == p &&
                (pid <= 0 || np->pid == pid)) {
                havekids = 1;
                if (np->state == ZOMBIE) {
```

```

        // 找到已结束的子进程
        np->state = UNUSED;
        *code = np->exit_code;
        return np->pid;
    }
}
}
if (!havekids) {
    return -1; // 没有子进程
}
// 子进程还在运行，让出CPU等待
p->state = RUNNABLE;
sched();
}
}

```

关键理解：

- wait 会阻塞父进程，直到子进程结束
- 子进程结束后进入ZOMBIE状态，保存exit_code
- 父进程wait时读取exit_code，然后释放子进程资源

3.5 fork + exec 的组合

为什么要把进程创建拆成两个系统调用？

指导书解释了原因：

"fork + exec 的组合是经过实践验证的灵活设计...可以方便地支持重定向、管道等高级功能。"

例如，Shell执行 `ls > output.txt` 时：

1. fork 创建子进程
2. 在子进程中，先重定向stdout到文件
3. 然后 exec 执行 ls

如果只有一个 spawn 调用，就无法在创建进程和执行程序之间插入重定向操作。

四、进程调度机制

4.1 从 FIFO 到 Stride

指导书提到，ch5框架使用队列实现FIFO调度：

```

struct queue {
    int data[QUEUE_SIZE];
    int front;
    int tail;
    int empty;
};

```


调度器 `fetch_task()` 从队列头取出下一个要运行的进程。

FIFO调度的问题：所有进程平等，无法支持优先级。

4.2 Stride 调度算法原理

本章编程作业要求实现 Stride 调度算法。

核心思想：

每个进程有两个属性：

- `stride`：累计的"虚拟运行时间"
- `priority`：优先级 (≥ 2)

调度规则：

1. 每次选择 `stride` 最小的进程运行
2. 运行后更新：`stride += BIG_STRIDE / priority`

直觉理解：

- 优先级高的进程，每次 `stride` 增加得少
- 所以它更容易被再次选中
- 结果是高优先级进程获得更多CPU时间

【原创结构图2：Stride调度示例图】

Stride调度示例图

轮次	A.stride	B.stride 选中	选中	更新后A.stride	更新后B.stride
初始	0	0	-	-	-
1	0	0	A	32768	0
2	32768	0	B	32768	16384
3	32768	16384	B	32768	32768
4	32768	32768	A	65536	32768
5	65536	32768	B	65536	49152
6	65536	49152	B	65536	65536

结论：A运行2次，B运行4次，符合优先级比例2:4=1:2
A: $\text{pass} = 65536 / 2 = 32768$, B: $\text{pass} = 65536 / 4 = 16384$

4.3 BIG_STRIDE 的选择

`BIG_STRIDE` 是一个常量，用于计算 `pass` 值。

选择原则：

1. 必须足够大，能被常见的优先级值整除（避免精度损失）

2. 不能太大，避免 stride 溢出

我们选择 `BIG_STRIDE = 65536 = 2^16`，可以被2到256范围内的大部分数整除。

五、Shell 工作原理分析

5.1 usershell.c 代码解读

现在我来仔细分析 `user/src/usershell.c` 的实现：

```
int main()
{
    printf("C user shell\n");
    printf(">> ");
    while (1) {
        char c = getchar(); // 读取一个字符
        switch (c) {
            case LF: // 换行符（回车）
            case CR:
                printf("\n");
                if (!is_empty()) {
                    push('\0'); // 字符串结尾
                    int pid = fork();
                    if (pid == 0) {
                        // 子进程：执行用户输入的程序
                        if (exec(line, NULL) < 0) {
                            printf("no such program: %s\n", line);
                            exit(0);
                        }
                    } else {
                        // 父进程：等待子进程结束
                        int xstate = 0;
                        waitpid(pid, &xstate);
                        printf("Shell: Process %d exited with code %d\n",
                               pid, xstate);
                    }
                }
                clear();
            }
            printf(">> ");
            break;
            case BS: // 退格键
            case DL:
                if (!is_empty()) {
                    putchar(BS);
                    printf(" ");
                    putchar(BS);
                    pop();
                }
                break;
            default: // 普通字符
                putchar(c); // 回显
                push(c);     // 保存
```

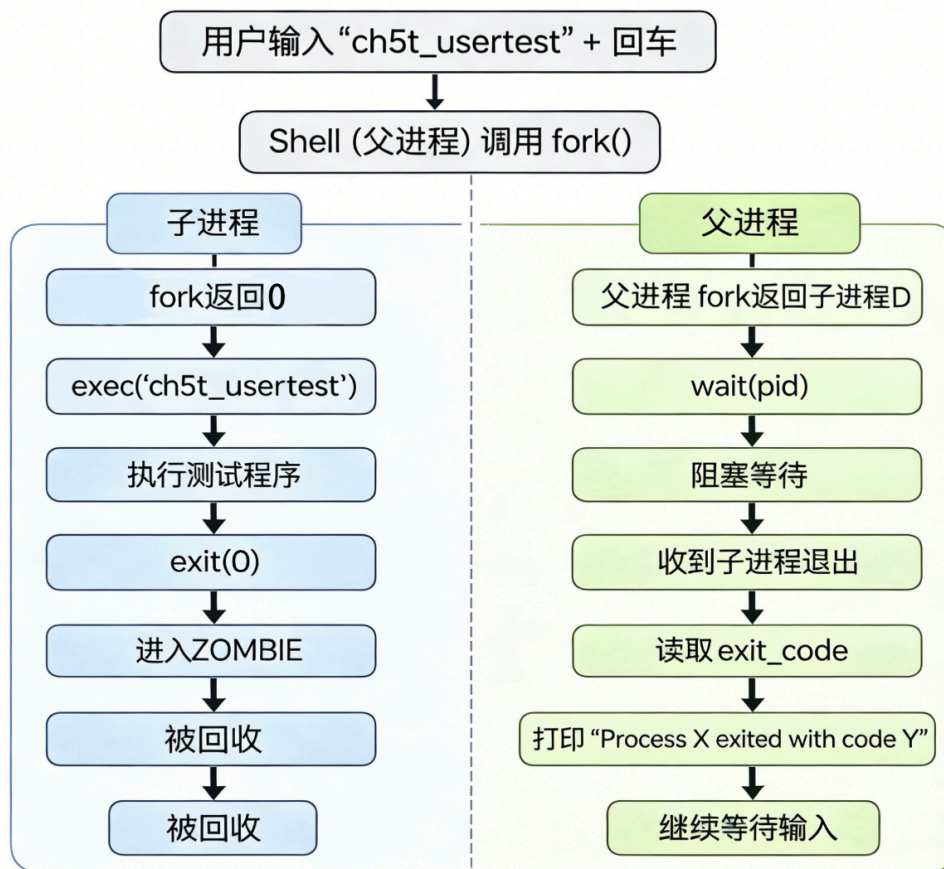
```

        break;
    }
}
return 0;
}

```

5.2 Shell 的工作流程

【原创结构图3: Shell工作流程图】



5.3 理解 `sys_read` 的作用

Shell 需要读取用户输入，这通过 `sys_read` 实现：

```

uint64 sys_read(int fd, uint64 va, uint64 len)
{
    if (fd != STDIN)
        return -1;
    struct proc *p = curr_proc();
    char str[MAX_STR_LEN];
    for (int i = 0; i < len; ++i) {
        int c = consgetc(); // 阻塞等待键盘输入
        str[i] = c;
    }
    copyout(p->pagetable, va, str, len);
    return len;
}

```

`consgetc()` 会阻塞等待，直到用户按下一个键。这就是为什么Shell会停在 `>>` 处等待输入。

六、编程作业实现

6.1 任务概述

本编程作业包含两个系统调用和一个调度算法：

功能	系统调用	调用号	说明
进程创建	<code>sys_spawn</code>	400	创建新进程并执行程序（无需复制父进程内存）
优先级设置	<code>sys_set_priority</code>	140	设置进程优先级
Stride调度	-	-	基于优先级的公平调度算法

6.2 理解 spawn vs fork+exec

传统 Unix 创建新进程的方式是 `fork+exec`：

`fork:` 父进程 → 子进程（完整复制内存）
`exec:` 子进程 → 执行新程序（丢弃刚复制的内存）

问题： `fork` 复制了整个地址空间，但 `exec` 立即丢弃它，这是浪费！

spawn 的优势： 直接创建新进程并加载程序，无需复制父进程内存。

6.3 sys_spawn 实现

```
/* ch5: spawn系统调用：创建新进程并执行程序 */
uint64 sys_spawn(uint64 va)
{
    struct proc *p = curr_proc();
    char name[200];
    copyinstr(p->pagetable, name, va, 200);
    debugf("sys_spawn %s\n", name);
    return spawn(name);
}

/* ch5: spawn系统调用：创建新进程并执行指定程序 */
/* ch5: 相当于 fork + exec，但不拷贝父进程内存，直接加载新程序 */
int spawn(char *name)
{
    struct proc *np;
    struct proc *p = curr_proc();

    /* ch5: 1. 分配新进程 */
    if ((np = allocproc()) == 0) {
        return -1; /* ch5: 进程池满或内存不足 */
    }
}
```

```

/* ch5: 2. 查找程序ID */
int id = get_id_by_name(name);
if (id < 0) {
    freeproc(np); /* ch5: 释放刚分配的进程 */
    return -1; /* ch5: 无效的文件名 */
}

/* ch5: 3. 加载程序到新进程（不拷贝父进程内存） */
loader(id, np);

/* ch5: 4. 设置父子关系 */
np->parent = p;

/* ch5: 5. 设置新进程为可运行状态并加入调度队列 */
np->state = RUNNABLE;
add_task(np);

/* ch5: 6. 返回子进程PID */
return np->pid;
}

```

6.4 sys_set_priority 实现

```

/* ch5: 设置进程优先级 */
/* ch5: 只要 prio 在 [2, isize_max] 就成功, 返回 prio, 否则返回 -1 */
uint64 sys_set_priority(long long prio){
    if (prio < 2) {
        return -1; /* ch5: 优先级必须 >= 2 */
    }

    struct proc *p = curr_proc();
    p->priority = (uint64)prio;

    debugf("sys_set_priority: pid=%d, new_priority=%lld\n", p->pid, prio);

    return prio;
}

```

6.5 Stride 调度算法实现

首先, 在进程控制块中添加stride和priority字段:

```

// os/proc.h
struct proc {
    // ... 原有字段 ...
    /* ch5: stride调度算法相关字段 */
    uint64 stride; /* ch5: 当前进程已运行的"stride长度" */
    uint64 priority; /* ch5: 进程优先级, 初始值为16 */
};

```

然后, 修改 `fetch_task` 函数实现 Stride 调度:

```

/* ch5: BIG_STRIDE常量, 用于stride调度算法 */
/* ch5: 因为测例中优先级都是2的整数次幂, 所以65536足够 */
#define BIG_STRIDE 65536

/* ch5: 基于stride调度算法选择下一个要运行的进程 */
/* ch5: 从就绪队列中选择stride最小的进程 */
struct proc *fetch_task()
{
    /* ch5: 遍历所有RUNNABLE进程, 找到stride最小的 */
    struct proc *min_stride_proc = NULL;
    uint64 min_stride = (uint64)-1; /* ch5: 最大值 */

    /* ch5: 遍历所有进程, 找到stride最小的RUNNABLE进程 */
    for (struct proc *p = pool; p < &pool[NPROC]; p++) {
        if (p->state == RUNNABLE) {
            if (p->stride < min_stride) {
                min_stride = p->stride;
                min_stride_proc = p;
            }
        }
    }

    if (min_stride_proc == NULL) {
        debug("No task to fetch\n");
        return NULL;
    }

    /* ch5: 更新选中进程的stride */
    /* ch5: pass = BIG_STRIDE / priority */
    uint64 pass = BIG_STRIDE / min_stride_proc->priority;
    min_stride_proc->stride += pass;

    /* ch5: 从就绪状态移除 (将在scheduler中设置为RUNNING) */
    min_stride_proc->state = RUNNING;

    debug("fetch task pid=%d, stride=%llu, priority=%llu\n",
          min_stride_proc->pid, min_stride_proc->stride, min_stride_proc->priority);

    return min_stride_proc;
}

```

6.6 进程初始化

在 `allocproc` 中初始化新字段:

```

struct proc *allocproc()
{
    // ... 分配进程 ...

    /* ch5: 初始化stride调度相关字段 */
    p->stride = 0;      /* ch5: 初始stride为0 */
    p->priority = 16;    /* ch5: 默认优先级为16 */

    return p;
}

```

6.7 遇到的问题与解决

问题1: ch5b_usertest 运行时出现页错误

现象:

```

[ERROR 2]13 in application, bad addr = 0x0000000010001ec8,
bad instruction = 0x0000000000002366, core dumped.

```

分析:

这与Lab4遇到的问题完全相同！原因是 `freewalk` 函数在遇到叶子页面时会 panic，而 mmap 分配的页面没有被正确清理。

解决: 修改 `freewalk` 函数，遇到叶子页面时释放它而不是 panic：

```

/* ch5: 修改为同时释放叶子页面，支持mmap区域的清理 */
void freewalk(pagetable_t pagetable)
{
    for (int i = 0; i < 512; i++) {
        pte_t pte = pagetable[i];
        if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
            // 指向下级页表
            uint64 child = PTE2PA(pte);
            freewalk((pagetable_t)child);
            pagetable[i] = 0;
        } else if (pte & PTE_V) {
            /* ch5: 释放叶子页面的物理内存，而不是panic */
            uint64 pa = PTE2PA(pte);
            kfree((void *)pa);
            pagetable[i] = 0;
        }
    }
    kfree((void *)pagetable);
}

```

问题2: add_task 函数不需要实际操作

在实现 Stride 调度时，我发现 `add_task` 函数可以保持空实现：

```
void add_task(struct proc *p)
{
    /* ch5: stride调度不需要使用队列，直接标记为RUNNABLE即可 */
    /* ch5: fetch_task会遍历所有RUNNABLE进程找到stride最小的 */
    debugf("add task pid=%d to ready queue\n", p->pid);
}
```

因为 `fetch_task` 是通过遍历进程池找 RUNNABLE 进程的，不依赖队列。

6.8 测试结果

```
C user shell
>> ch5t_usertest
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
ch5t usertest passed!
Shell: Process 2 exited with code 0
>>
```

所有 Stride 调度测试通过！

七、新的程序加载机制

7.1 从 run_all_app 到 load_init_app

在之前的章节中，`run_all_app` 函数会在内核启动时加载并运行所有应用程序。但从本章开始，这个函数被 `load_init_app` 取代：

```
int load_init_app()
{
    int id = get_id_by_name(INIT_PROC); // INIT_PROC = "usershell"
    if (id < 0)
        panic("Cannot find INIT_PROC %s", INIT_PROC);
    struct proc *p = allocproc();
    if (p == NULL) {
        panic("allocproc\n");
    }
    loader(id, p);
    return 0;
}
```

现在内核只加载一个初始程序——usershell，由它来负责运行其他程序。

7.2 为什么要复制程序镜像？

在 Lab4 中，`bin_loader` 直接将虚拟地址映射到程序的物理镜像。但 Lab5 改为复制：

```
for (uint64 va = va_start, pa = pa_start; pa < pa_end;
    va += PGSIZE, pa += PGSIZE) {
    page = kalloc();
    memmove(page, (const void *)pa, PGSIZE); // 复制内容
    mappages(p->pagetable, va, PGSIZE, (uint64)page, ...);
}
```

为什么要复制？

因为现在程序可以被多次执行！

- Lab4：每个程序只运行一次，修改原始镜像没关系
- Lab5：用户可以多次运行同一个程序，如果修改了原始镜像，第二次运行就会出错

复制镜像确保每次运行都从干净的初始状态开始。

八、本章新增系统调用汇总

系统调用	调用号	功能	来源
<code>sys_read</code>	63	从标准输入读取	框架提供
<code>sys_write</code>	64	输出字符串	Lab2 已有
<code>sys_exit</code>	93	退出进程	Lab2 已有
<code>sys_sched_yield</code>	124	主动让出 CPU	Lab3 已有
<code>sys_set_priority</code>	140	设置进程优先级	本章作业
<code>sys_gettimeofday</code>	169	获取当前时间	Lab3 已有
<code>sys_getpid</code>	172	获取进程ID	框架提供
<code>sys_getppid</code>	173	获取父进程ID	框架提供
<code>sys_clone (fork)</code>	220	创建子进程	框架提供
<code>sys_execve</code>	221	执行新程序	框架提供
<code>sys_wait4</code>	260	等待子进程	框架提供
<code>sys_spawn</code>	400	创建新进程并执行程序	本章作业

九、实验总结

完成情况

- ✓ 理解交互式Shell的意义和工作原理
- ✓ 理解进程状态转换，特别是ZOMBIE状态
- ✓ 理解fork"一次调用两次返回"的原理
- ✓ 理解exec和wait系统调用
- ✓ 理解Stride调度算法的原理
- ✓ 实现sys_spawn系统调用
- ✓ 实现sys_set_priority系统调用
- ✓ 实现Stride调度算法
- ✓ 修复freewalk以支持mmap区域清理
- ✓ 通过所有测试

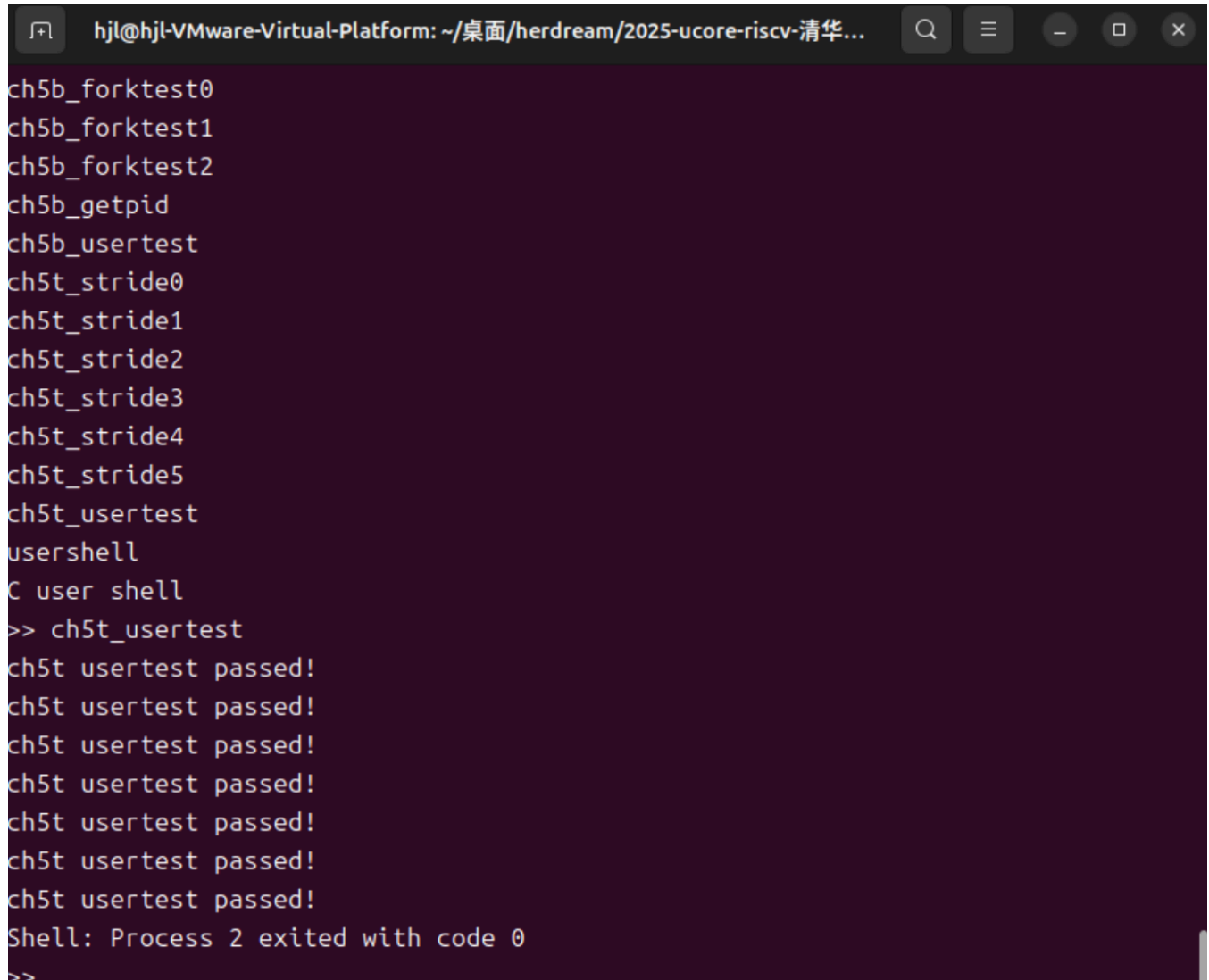
收获与体会

- 交互式操作是巨大的进步：**从"观众"变成"操作者"，这让操作系统真正变得可用。Shell虽然简单，但它是所有交互的基础。
- fork的设计很巧妙：**通过修改trapframe中的a0寄存器，让同一个系统调用在父子进程中返回不同的值。这种设计既简单又优雅。
- Stride调度的数学之美：**通过简单的除法和加法，就能保证进程按优先级比例获得CPU时间。数学直觉（优先级高→增量小→更容易被选中）非常清晰。
- 问题会跨章节延续：**freewalk的panic问题在ch4就应该修复，但由于当时测试不充分，问题延续到了ch5。这提醒我在实现功能时要考虑更多边界情况。
- 理解"为什么"比"怎么做"更重要：**比如理解为什么要fork+exec两步而不是一个spawn调用，为什么要复制程序镜像而不是直接映射，这些设计决策背后都有深刻的原因。

文件修改清单

文件	修改内容
os/proc.h	添加 stride 和 priority 字段，添加 spawn() 函数声明
os/proc.c	初始化stride/priority，实现 fetch_task() 的Stride调度，实现 spawn()
os/syscall.c	添加 sys_spawn() 和 sys_set_priority() 入口
os/vm.c	修改 freewalk() 支持释放叶子页面

十、验证截图



```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华...  
ch5b_forktest0  
ch5b_forktest1  
ch5b_forktest2  
ch5b_getpid  
ch5b_usertest  
ch5t_stride0  
ch5t_stride1  
ch5t_stride2  
ch5t_stride3  
ch5t_stride4  
ch5t_stride5  
ch5t_usertest  
usershell  
C user shell  
>> ch5t_usertest  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
Shell: Process 2 exited with code 0  
>>
```

关键输出：

```
C user shell  
>> ch5t_usertest  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
ch5t usertest passed!  
Shell: Process 2 exited with code 0
```

所有 ch5 相关测试通过，说明 spawn 和 Stride 调度实现正确。